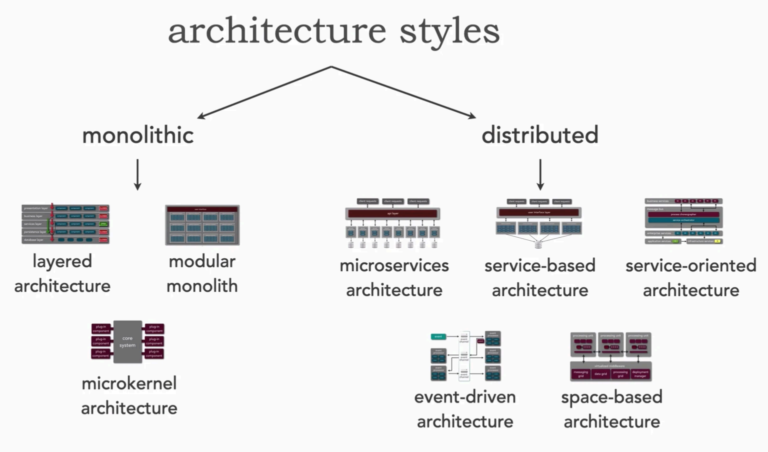
Software Architectures

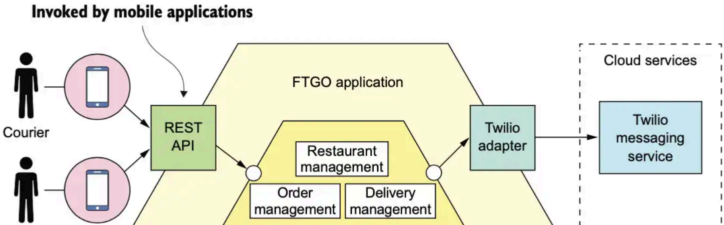
What is Software Architecture?

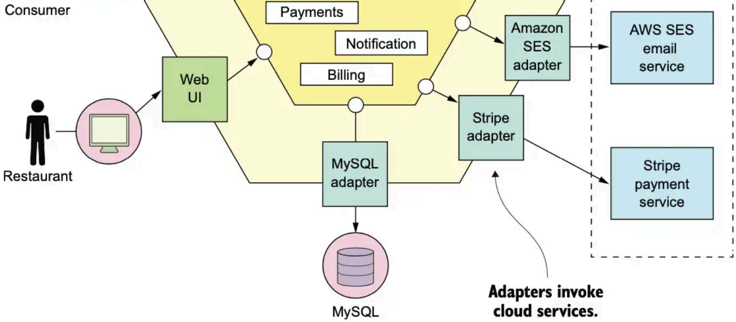
Architecture defines **components, connectors, and constraints** that shape how systems are organized. Different styles change **local complexity** (inside modules) and **global complexity** (system-wide).


architecture styles
monolithic
distributed
layered architecture
modular monolith
microkernel architecture
microservices architecture
service-based architecture
service-oriented architecture
event-driven architecture
space-based architecture

Monolithic Architectures

A **single, unified application** where all parts run in one process.


Invoked by mobile applications
Courier
REST API
FTGO application
Restaurant management
Order management
Delivery management
Twilio adapter
Cloud services
Twilio messaging service

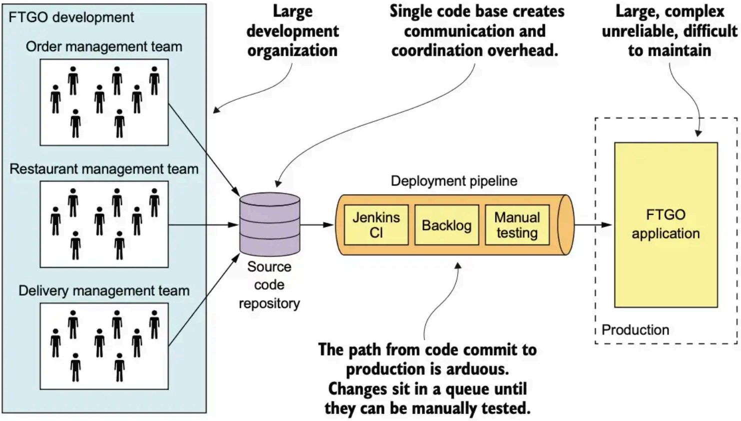

Consumer
Payments
Notification
Billing
Web UI
Amazon SES adapter
AWS SES email service
Stripe adapter
Stripe payment service
MySQL adapter
MySQL
Restaurant
Adapters invoke cloud services.

Pros

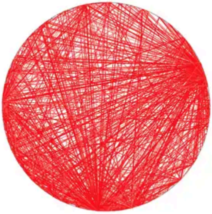
- Simple development, testing, deployment
- Fast in-process communication

Cons (when project grows)

- Tends toward a *big ball of mud*
- Slow development loops (startup, testing, deployment)
- Hard to refactor
- Tech lock-in
- Can't scale or deploy individual pieces

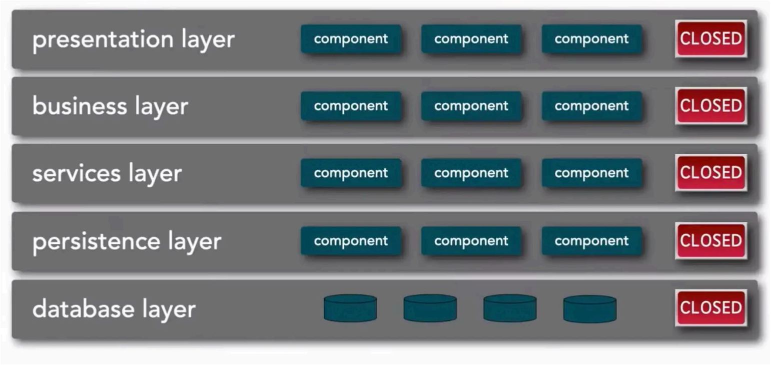

FTGO development
Order management team
Restaurant management team
Delivery management team
Large development organization
Single code base creates communication and coordination overhead.
Large, complex unreliable, difficult to maintain
Source code repository
Deployment pipeline
Jenkins CI
Backlog
Manual testing
FTGO application
Production
The path from code commit to production is arduous. Changes sit in a queue until they can be manually tested.

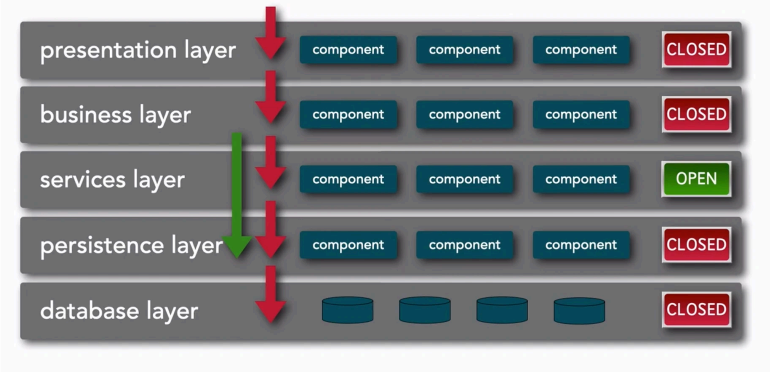
Big Ball of Mud



Multi-Layered (N-tier) Architecture

Application split into technical layers (UI → business → data).


presentation layer
component
component
component
CLOSED
business layer
component
component
component
CLOSED
services layer
component
component
component
CLOSED
persistence layer
component
component
component
CLOSED
database layer
CLOSED


presentation layer
component
component
component
CLOSED
business layer
component
component
component
CLOSED
services layer
component
component
component
OPEN
persistence layer
component
component
component
CLOSED
database layer
CLOSED

Pros

- Clear separation of concerns
- Easy for teams to work on different layers
- Familiar and well-supported

Cons

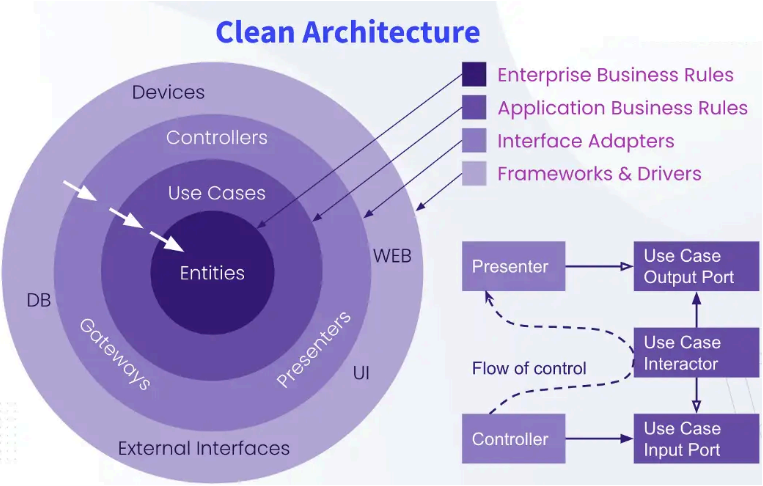
- Vertical features require changes across layers
- Inefficient message flow through layers
- Harder to refactor around domain boundaries

Use

- Simple applications
- Technically structured teams

Clean Architecture

Domain and use-cases at the center; frameworks and infrastructure at the edges.


Clean Architecture
Enterprise Business Rules
Application Business Rules
Interface Adapters
Frameworks & Drivers
Devices
Controllers
Use Cases
Entities
WEB
DB
Gateways
Presenters
UI
External Interfaces
Presenter
Use Case Output Port
Use Case Interactor
Flow of control
Controller
Use Case Input Port

Pros

- Strong decoupling
- Easy to test business logic
- Swap DB/UI/frameworks without touching core logic

Cons

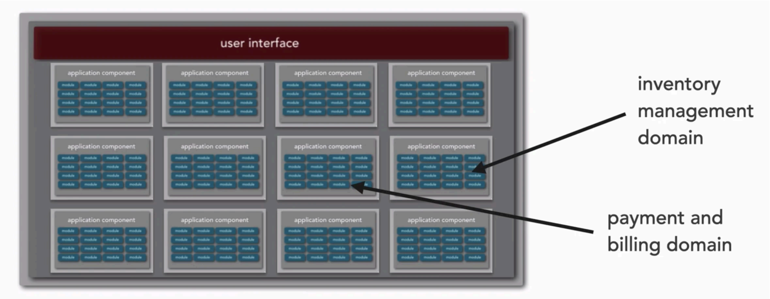
- More abstraction and indirection
- Requires architectural discipline

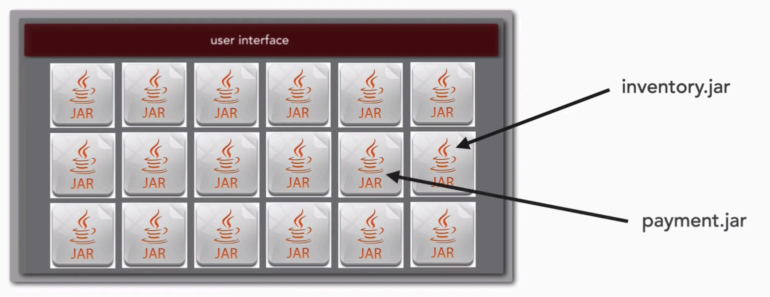
Use

- Systems expected to evolve for years
- Applications with complex business rules

Modular Monolithic Architecture

Organize code by **business modules**, not layers.


user interface
application component
module
inventory management domain
payment and billing domain


user interface
JAR
inventory.jar
payment.jar

Pros

- Strong domain encapsulation
- Good consistency (single database)
- Natural path toward microservices

Cons

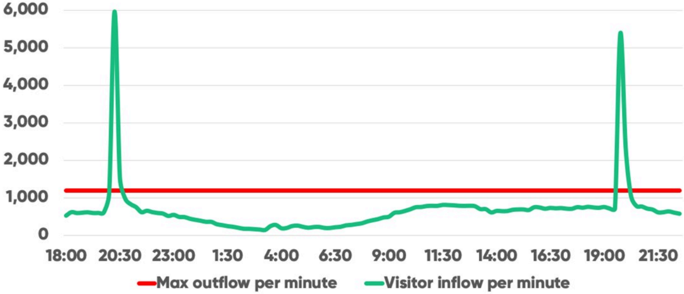
- No independent deployment
- No technology polyglot
- Scaling still applies to entire app

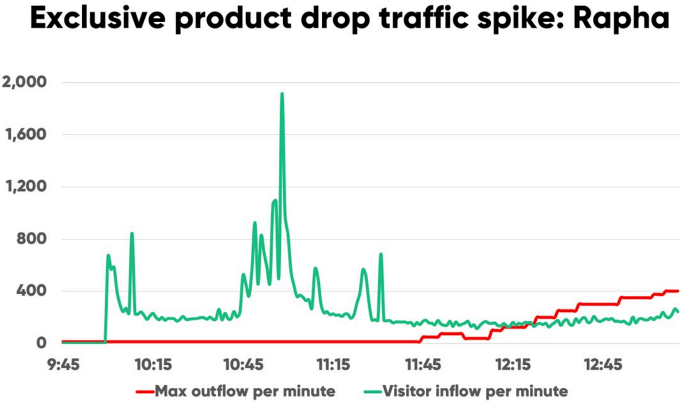
Use

- Domain-oriented teams
- Large monoliths undergoing modernization
- Workloads requiring strong consistency

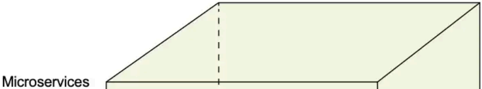
Monoliths and the Challenge of Traffic Spikes

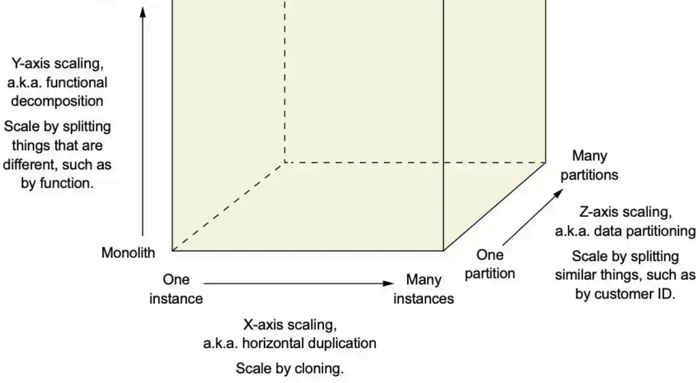
PR appearance traffic spike: Rakuten France




Exclusive product drop traffic spike: Rapha

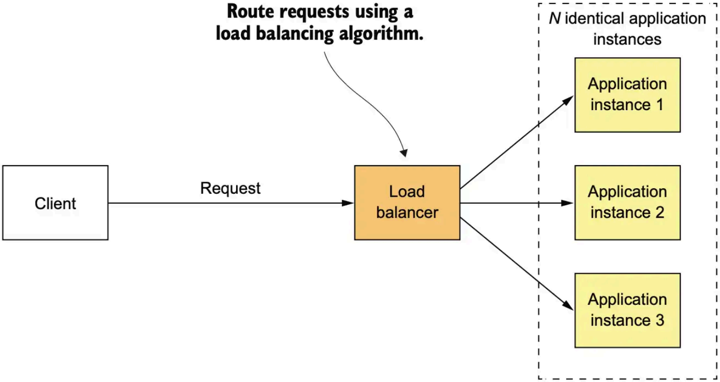
Scaling Monoliths — The Scale Cube


Microservices


Y-axis scaling, a.k.a. functional decomposition
Scale by splitting things that are different, such as by function.
Monolith
One instance
Many instances
X-axis scaling, a.k.a. horizontal duplication
Scale by cloning.
One partition
Many partitions
Z-axis scaling, a.k.a. data partitioning
Scale by splitting similar things, such as by customer ID.

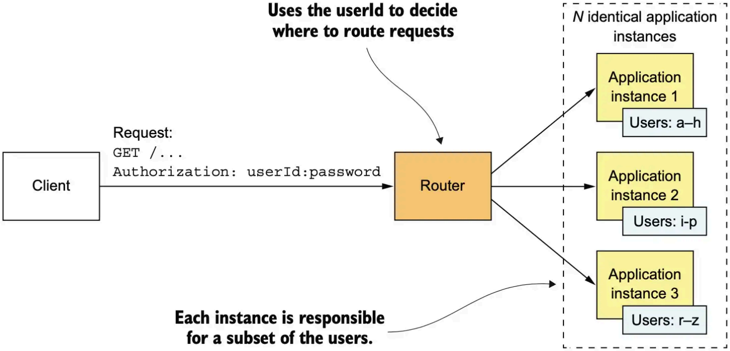
X-axis Scaling

Clone the entire application behind a load balancer.


Route requests using a load balancing algorithm.
N identical application instances
Client
Request
Load balancer
Application instance 1
Application instance 2
Application instance 3

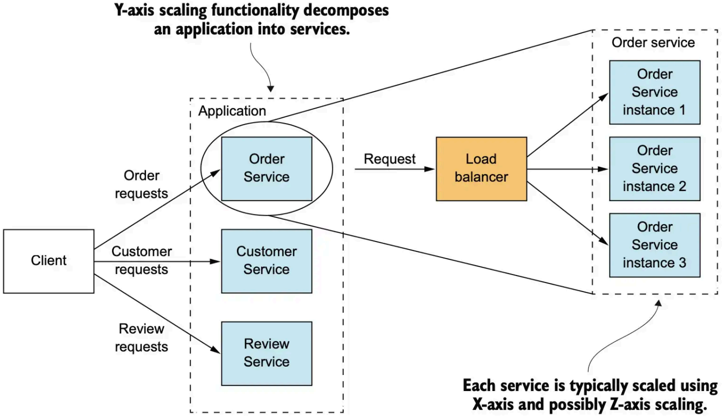
Z-axis Scaling

Shard by data (e.g., userId).


Uses the userId to decide where to route requests
N identical application instances
Client
Request:
GET /...
Authorization: userId:password
Router
Application instance 1
Users: a–h
Application instance 2
Users: i-p
Application instance 3
Users: r–z
Each instance is responsible for a subset of the users.

Y-axis Scaling

Split the application by function.


Y-axis scaling functionality decomposes an application into services.
Application
Order Service
Customer Service
Review Service
Client
Order requests
Customer requests
Review requests
Request
Load balancer
Order service
Order Service instance 1
Order Service instance 2
Order Service instance 3
Each service is typically scaled using X-axis and possibly Z-axis scaling.

Distributed Architectures

Applications composed of multiple components running on different machines, communicating over RPC.

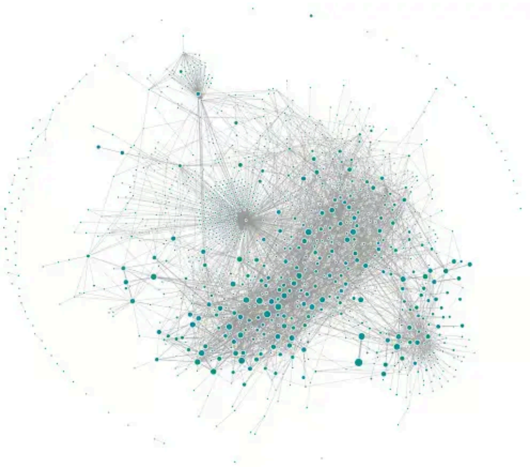
Pros

- Independent deployment
- Independent scaling
- Smaller units → lower local complexity

Cons

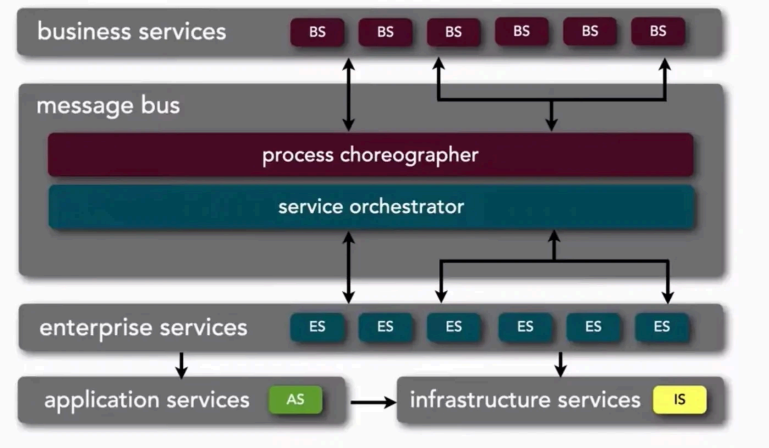
- High global complexity (APIs, retries, consistency)
- Requires service discovery, tracing, logging

- Risk of a *distributed monolith*



Service-Oriented Architecture (SOA)

Dumb services, smart pipes. Often built around shared enterprise services and protocols like SOAP.


business services
BS
BS
BS
BS
BS
BS
message bus
process choreographer
service orchestrator
enterprise services
ES
ES
ES
ES
ES
ES
application services
AS
infrastructure services
IS

Pros

- Good for integrating heterogeneous systems
- Reusable, discoverable services

Cons

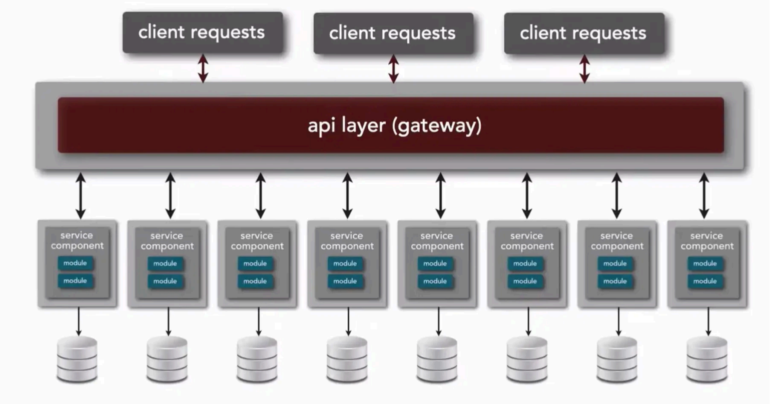
- Heavy protocols
- Shared services become bottlenecks
- Slow adaptation to change

Use

- Large enterprises with legacy systems
- Cross-system workflows

Microservices Architecture

Smart services, dumb pipes. Each service owns its data and is deployed independently.


client requests
client requests
client requests
api layer (gateway)
service component
module
module

Pros

- High evolvability
- Independent deploy and scale
- Technology freedom
- Team autonomy

Cons

- Operational complexity
- Harder debugging and monitoring
- Many small services/databases to manage

Use

- Complex systems with fast evolution
- Teams experienced in distributed systems

Resources

- *Fundamentals of Software Architecture* — Richards & Ford
- Mark Richards – YouTube channel